

Current State of RAG (Production Failure Modes) — Interview Questions

Reproduced exactly as originally written from Retrieval Augmented Generation (RAG)/Current State of RAG/Current State of RAG - Video Notes.md — nothing reworded.

This session's real focus was production reliability, not RAG mechanics — so this interview set is deliberately different from the RAG Workflow and Vector Database sets: it's about failure modes, trade-offs, and what breaks a RAG system once it leaves the demo stage. Same two-layer format as those sections: a plain-language answer with an example, then a crisp, technically precise version. Try answering out loud first.

 **Interview Q1:** "If each stage of your RAG pipeline is individually 90% reliable, why isn't the overall system 90% reliable?"

✅ **Strong answer:** "Because reliability *multiplies* across stages, it doesn't average. Think of it like a relay of six people each passing a message along with a 90% chance of passing it correctly — the message only survives if *every single* handoff succeeds. Six handoffs at 90% each gives you $0.9 \times 0.9 \times 0.9 \times 0.9 \times 0.9 \times 0.9 \approx 53\%$. This is literally how the session's own architecture diagram illustrates it: ingestion, chunking, embedding, index, retrieval, and re-ranking, each at 90%, multiply down to 53% end-to-end reliability — even though no single stage looks broken in isolation."

🎯 **Standard Interview Answer:** "Pipeline reliability compounds multiplicatively across independent stages: $\text{overall_reliability} \approx \prod(\text{stage_reliability})$. At 90% per stage across six stages (ingestion, chunking, embedding, index, retrieval, re-ranking), the system-level reliability is approximately $0.9^6 \approx 53\%$. This is why per-stage evaluation is necessary but not sufficient — a system can look healthy stage-by-stage while the compounded end-to-end answer quality is mediocre, which is exactly why isolated component metrics without an end-to-end eval can be misleading."

 **Interview Q2:** "What's the actual failure mode with fixed-size chunking, and why doesn't 'just use a bigger chunk size' fix it?"

✅ **Strong answer:** "Fixed-size chunking splits by character or token count with no regard for meaning, so a single idea can get sliced in half — half a sentence in one chunk, the rest in the next, both now missing context the other half had. Making chunks bigger doesn't fix this cleanly either: too big and you run into the 'lost in the middle' problem, where the model pays less attention to information sitting in the middle of a long chunk than at its edges. **Example:** a paragraph explaining a company's refund *policy*, immediately followed by the *exception* to that policy, gets cut at a fixed character mark — the retrieved chunk only contains the policy, not its exception, and the model confidently gives an incomplete answer."

🎯 **Standard Interview Answer:** "Fixed-size (character or token count) chunking ignores semantic boundaries, fragmenting coherent ideas across chunk edges and degrading retrieval precision even when the embedding and index are otherwise sound. Increasing chunk size doesn't resolve this because it trades one failure mode for another — larger chunks dilute the embedding's specificity and trigger the 'lost in the middle' effect during generation, where attention to mid-context tokens degrades relative to context-edge tokens. Semantic chunking, splitting on meaning boundaries rather than a fixed count, addresses the fragmentation problem directly, at the cost of requiring an additional embedding pass at ingestion time."

 **Interview Q3:** "A user searches for 'heart attack' but the source documents say 'myocardial infarction.' Walk me through why this fails, and how you'd fix it."

✅ **Strong answer:** "This is a vocabulary mismatch between how users talk and how documents are written — plain-English search terms versus clinical or domain-specific terminology. A pure keyword search fails outright, since 'heart attack' and 'myocardial infarction' share no words. A pure dense/semantic search often *does* catch this, since the embeddings for both phrases land close together in meaning-space — but semantic search alone can also miss cases where an exact keyword match genuinely mattered. The standard fix is hybrid search: run both a keyword search and a semantic search, then merge and de-duplicate the two result sets with Reciprocal Rank Fusion, so you're covered whichever kind of match the query actually needed."

🎯 **Standard Interview Answer:** "This is a lexical–semantic vocabulary gap between query and corpus language. Sparse retrieval (BM25) fails on exact-term mismatch since there's no token overlap; dense retrieval typically succeeds here since embedding models capture the synonymy between colloquial and clinical terms. The standard mitigation is hybrid retrieval — BM25 plus dense vector search, fused via Reciprocal Rank Fusion — which covers both failure directions. In practice, off-the-shelf RRF or re-ranking models often still need domain-specific tuning to perform well on a specialized vocabulary like clinical terminology, since a general-purpose fusion algorithm has no inherent knowledge of that domain's language patterns."

 **Interview Q4:** "What's wrong with using a fixed top-K for retrieval, and what would you do instead?"

✅ **Strong answer:** "A fixed K is a one-size-fits-all guess that's wrong for most queries. Set K too low, and a genuinely relevant chunk that would've been the 4th-best match never makes it in if K is 3 — insufficient context. Set K too high, and a simple query pulls in a pile of marginally-relevant chunks that just add noise, cost, and latency without adding anything useful — excessive context. The fix is dynamic K: size K to how hard or broad the query actually is — a simple, narrow question gets a small K, a complex or multi-part question gets a larger one."

🎯 **Standard Interview Answer:** "Fixed top-K is a static heuristic applied uniformly regardless of query complexity, producing either insufficient context (K too low for a query whose relevant evidence spans more chunks) or excessive, noisy context (K too high for a simple query, increasing token cost, latency, and irrelevant-content dilution). Dynamic K — sized via a score threshold or a model-estimated query-difficulty signal — adapts retrieval depth to the query. The trade-off is that difficulty estimation is itself model-dependent and requires careful threshold calibration; a poorly calibrated difficulty estimator can just relocate the fixed-K problem rather than solve it."

 **Interview Q5:** "Why are multi-hop questions still an unsolved problem in RAG, even with modern reasoning models?"

✅ **Strong answer:** "A multi-hop question needs evidence stitched together from *several different* chunks, documents, or pages — not one single passage that already contains the full answer. Standard retrieve-once-and-generate can't do this, because a single similarity search just isn't built to chase down a chain of connected facts. The workaround is iterative or agentic retrieval: retrieve something, reason about whether it's enough, retrieve again if it isn't, and repeat — but every extra retrieval 'hop' adds real latency, and you need a clear rule for when to stop trying, or the loop can run forever."

🎯 **Standard Interview Answer:** "Multi-hop questions require synthesizing evidence distributed across multiple non-adjacent chunks or documents, which a single-pass similarity search cannot resolve, since it optimizes for finding chunks similar to the *original* query, not for chaining intermediate facts. Reasoning models mitigate this partially through iterative or agentic retrieval — retrieve, reason over sufficiency, re-query, repeat — but this

incurs compounding latency per hop and requires an explicit termination policy (a hop cap or a confidence threshold) to bound worst-case latency and avoid unbounded loops. This remains an open problem: low-latency multi-hop reasoning at production scale is not yet solved."

 **Interview Q6:** "What is p99 latency, and why would you track it broken down by different segments of your system rather than as one overall number?"

✅ **Strong answer:** "p99 latency is the response time below which 99% of requests complete — only the worst 1% are slower than that number, and it matters more than an average because an average can look perfectly healthy while a real slice of users are having a genuinely bad experience. **A real example makes this concrete:** a system's *average* latency might be 350 milliseconds — which looks great — while its *p99* latency is 2.8 seconds. That average was quietly hiding a genuinely bad experience for the worst 1% of requests. The reason to break p99 down by segment — say, by which vector index got hit, or which document category a query fell into — is debugging speed: if you only track one overall p99 number and a customer complains about slowness, you have no idea *where* to look. If you're tracking p99 per category, and one specific category suddenly spikes while the others stay flat, you know exactly which piece of the system to debug first."

🎯 **Standard Interview Answer:** "p99 latency is the 99th-percentile response time — 99% of requests complete at or below this value, with only the tail 1% exceeding it; it's tracked instead of (or alongside) mean latency because averages mask tail behavior that directly affects real user experience. Percentile-based SLOs are typically defined across the whole distribution — for example $P50 \leq 400\text{ms}$, $P90 \leq 900\text{ms}$, $P95 \leq 1.5\text{s}$, $P99 \leq 2.5\text{s}$ — and validated with load testing at realistic, escalating concurrency (e.g., ramping from 50 up to 500 simulated users), not just a single-user benchmark. Segmenting p99 by dimension (vector index, document category, single-index vs. multi-index query, retrieval vs. generation time) supports fast root-cause attribution: a regression isolated to one segment's p99 immediately narrows the debugging surface, versus a single aggregate p99 regression that could originate anywhere in the pipeline."

💬 **Interview Q6 (follow-up):** "If you're describing a RAG system and considering p99, explicitly tell me what categories we should be looking for."

✅ **Strong answer:** "Two separate axes, and they answer different debugging questions."

By pipeline stage (tells you *where* the time goes): query understanding/rewriting, retrieval, reranking, context assembly, and generation.

By workload/data segment (tells you *which requests* are slow): which vector index or collection was searched, document category or domain (e.g. legal queries vs. support-ticket queries), query type (simple lookup vs. summarization vs. multi-hop), tenant/customer in a multi-tenant system, and query complexity (single-hop vs. multi-hop chains).

A healthy per-stage breakdown with one bad workload segment, or vice versa, are two completely different bugs that look identical in a single overall p99 number — which is exactly why both axes need their own tracking, not just one aggregate figure."

 **Interview Q7:** "Why is it a mistake to evaluate retrieval quality and generation quality completely separately?"

✅ **Strong answer:** "Because the two failures look identical from the outside — a bad final answer — but need completely different fixes depending on which stage actually broke. If retrieval pulled the right chunk and the answer is still wrong, that's a hallucination problem in generation. If retrieval never found the right chunk in the

first place, generation never had a chance, and the fix belongs entirely upstream. If you only check 'is retrieval good' and 'is generation good' as two separate, disconnected checks, you can miss cases where each one looks fine in isolation, but the *combination* — the exact chunk generation actually received for that exact query — was the real problem."

 **Standard Interview Answer:** "Retrieval and generation failures are observationally identical at the output level (a wrong final answer) but require disjoint remediation paths. Evaluating each independently is necessary — did retrieval return the right evidence, did generation faithfully use it — but insufficient on its own, because it can miss failures specific to the retrieval-generation *interaction* for a given query, cases that only surface when you inspect what was actually retrieved *for* what was actually generated, together, rather than as two separately-scored pipeline components with separate benchmark sets."

 **Interview Q8:** "Besides the algorithm itself, what are the three things a production RAG system should never skip?"

 **Strong answer:** "Hybrid retrieval with re-ranking as the baseline search mechanism, metadata-rich chunking combined with hierarchical indexing, and an evaluation-first design — building the test set and per-component evaluation before you trust anything else. Skipping any of these three is where a lot of real systems land at surprisingly low accuracy — think 30-50%, not the 90%+ people assume RAG delivers by default."

 **Standard Interview Answer:** "The three non-negotiable baseline components are: (1) hybrid retrieval — sparse plus dense search fused via RRF — combined with a cross-encoder re-ranking stage, (2) metadata-rich chunking paired with hierarchical indexing for precise filtering and drill-down, and (3) evaluation-first design — a real test set with retrieval, generation, and end-to-end evaluation, iterated on, rather than assumed. Omitting any of these three is empirically associated with production RAG systems landing well below 70% accuracy, since the compounding-failure math from Q1 means a single unmonitored, un-augmented stage can silently anchor the whole pipeline's ceiling."

 **Interview Q9:** "Your team benchmarked a RAG system at under two minutes response time in dev, but stakeholders are seeing five minutes in production. What happened, and how do you prevent this?"

 **Strong answer:** "This is almost always a gap between the dev benchmark's conditions and real production conditions — smaller test corpus, no real concurrent traffic, cached warm-up runs, or a narrower slice of query types than production actually sees. The fix isn't really a *technical* fix so much as a *process* one: load-test against production-realistic data volume and concurrency *before* release, not after stakeholders start complaining — and measure p99, not just a best-case or average run, since that's exactly the number that exposes this kind of gap before it becomes a live incident."

 **Standard Interview Answer:** "This pattern typically indicates that development benchmarking wasn't representative of production load — smaller corpus size, no concurrent request contention, unrealistic cache-hit rates, or a narrower query distribution than production traffic actually exhibits. The structural fix is shifting load testing earlier: benchmark against production-scale data volume, concurrency, and query diversity pre-release, and report p99 (not mean) latency as the primary SLA metric, since mean latency is precisely the metric that hides this class of regression until it surfaces as a stakeholder-visible production incident."

Sources consulted while calibrating this section: - [RAG Interview Questions \(2026\): The Complete Guide — GitGood](#) - [Production RAG: 5 Failure Modes We Keep Seeing — Tensoria](#) - [RAG Accuracy Problems: Why They Happen and How to Fix Them — Atlan](#) - [Production RAG: Why Retrieval Fails and How to Fix It - LLM Evaluation](#)

Interview Questions: Evals, LLM-as-Judge, Drift, and Production Quality — PracHub - Semantic Observability:
Engineering Reliability for Production RAG — DEV Community - Latency as Evaluation Dimension — Ragas
Advanced Course, The Neural Base